Exploratory Data Analysis

of Netflix Dataset

ENTC A2 — Academic Project Report

NEHA BHAT — 25070123032
DHVAN SHAH — 25070123042
ATHARVA JADHAV — 25070123027
AYUSH PATHARE — 25070123166

1. Introduction

Exploratory Data Analysis (EDA) is the critical first step in any data science project. It is a systematic approach to understanding, summarizing, and visualizing a dataset before applying machine learning models. Rather than jumping directly into modeling, EDA helps data scientists discover patterns, detect anomalies, test hypotheses, and verify assumptions through statistical summaries and visualizations.

In this project, we perform a comprehensive EDA on the Netflix Titles Dataset — a publicly available dataset containing metadata about movies and TV shows available on Netflix. The dataset provides a rich source of categorical, temporal, and textual data, making it ideal for exploring a wide variety of analysis techniques.

1.1 Objectives

- Understand the structure, shape, and data types of the Netflix dataset
- Detect and handle missing values, outliers, and inconsistencies
- Uncover patterns in content type, ratings, countries, and release trends
- Apply feature engineering to derive new analytical features
- Demonstrate use of Python data science libraries: Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn
- Visualize findings through diverse chart types across Experiments 16, 17, 19, and 20

1.2 Key Terms Glossary

EDA	Exploratory Data Analysis — a process of analyzing datasets to summarize main characteristics using visual and statistical methods
DataFrame	A 2-dimensional tabular data structure in Pandas with labeled rows and columns, similar to a spreadsheet
Missing Values	NaN (Not a Number) entries in a dataset where data was not recorded or is unavailable
Feature Engineering	The process of creating new variables (features) from existing data to improve analysis or model performance
Label Encoding	Converting categorical text variables into integer numbers so they can be used in ML algorithms
Univariate Analysis	Analysis of a single variable to understand its distribution and summary statistics
Bivariate Analysis	Analysis of the relationship between two variables, e.g., correlation or cross-tabulation
Outlier	A data point that is significantly different from other observations — may indicate errors or interesting findings
Skewness	A measure of asymmetry in a distribution — positive skew means the tail is on the right
Heatmap	A color-coded matrix visualization used to show correlation or intensity values between variables

2. Netflix Dataset Overview

The Netflix Titles Dataset contains approximately 8,800 records describing movies and TV shows available on the Netflix streaming platform. It was sourced from Kaggle and captures Netflix's global content library.

2.1 Dataset Schema

Column	Data Type	Description
show_id	String	Unique identifier for each title (e.g., s1, s2, ...)
type	Categorical	"Movie" or "TV Show" — the primary classification
title	String	Name of the movie or TV show
director	String	Name(s) of the director(s) — has many missing values
cast	String	Comma-separated list of actors featured
country	String	Country/countries where the content was produced
date_added	Date String	Date the title was added to Netflix (e.g., "January 1, 2020")
release_year	Integer	Year the title was originally released to the public
rating	Categorical	Content rating (TV-MA, TV-14, R, PG-13, etc.)
duration	String	Runtime — "90 min" for movies or "2 Seasons" for shows
listed_in	String	Genre categories (e.g., "Dramas, International Movies")
description	String	Short synopsis of the title

2.2 Initial Data Inspection Code

The following code was used at the start of the project to load the dataset and perform initial inspection:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv("netflix_titles.csv")

# Basic inspection
print(df.shape)           # (8807, 12) — rows and columns
print(df.dtypes)          # data types of each column
print(df.isnull().sum())  # count missing values per column
print(df.head())          # first 5 rows
print(df.describe())      # summary statistics for numeric columns
```

Key findings from initial inspection:

- Shape: ~8,807 rows × 12 columns
- Missing values found in: director (~30%), cast (~10%), country (~7%), date_added (~0.1%), rating (~0.04%)
- release_year is the only numeric column — all others are categorical or text

3. Data Preprocessing & Scikit-learn

Data preprocessing is the process of transforming raw data into a clean and structured format suitable for analysis and machine learning. It is one of the most critical steps — poor data quality leads directly to unreliable results.

3.1 Handling Missing Values

```
# Check for missing values
print(df.isnull().sum())

# Fill missing categorical columns with "Unknown"
df["director"].fillna("Unknown", inplace=True)
df["cast"].fillna("Unknown", inplace=True)
df["country"].fillna("Unknown", inplace=True)

# Drop rows where "rating" or "date_added" is missing (very few)
df.dropna(subset=["rating", "date_added"], inplace=True)
```

fillna() replaces NaN values with a specified value. **dropna(subset=[...])** removes rows only where specific columns are NaN, preserving as much data as possible.

3.2 Converting Date Columns

```
# Convert "date_added" from string to datetime
df["date_added"] = pd.to_datetime(df["date_added"].str.strip(), errors="coerce")

# Verify the conversion
print(df["date_added"].dtype)  # datetime64[ns]
print(df["date_added"].max())  # most recent date in dataset
```

pd.to_datetime() converts string dates into Python datetime objects, enabling extraction of year, month, and day components. **str.strip()** removes leading/trailing whitespace that can cause parsing errors. The **errors="coerce"** parameter converts unparseable strings to NaT (Not a Time) instead of raising an exception.

3.3 Feature Engineering

Feature engineering creates new columns derived from existing data. These derived features often carry more analytical and predictive value than the original raw columns.

```
# Extract year and month from date_added
df["year_added"]  = df["date_added"].dt.year
df["month_added"] = df["date_added"].dt.month

# Calculate content age: how old is the content relative to 2024
df["content_age"] = 2024 - df["release_year"]

# Extract numeric duration for movies
movies = df[df["type"] == "Movie"].copy()
movies["duration_min"] = movies["duration"].str.extract(r"(\d+)").astype(float)
```

Three key engineered features were created:

- year_added — enables year-over-year trend analysis of Netflix content additions
- month_added — reveals seasonal patterns (summer peaks, holiday spikes)
- content_age — measures how old content is when Netflix acquired it

3.4 Label Encoding with Scikit-learn

Machine learning algorithms cannot process text directly — they require numerical input. Label Encoding transforms categorical text labels into integer representations.

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

# Encode "type" column: Movie → 0, TV Show → 1
df["type_encoded"] = le.fit_transform(df["type"])

# Encode "rating" column
df["rating_encoded"] = le.fit_transform(df["rating"].astype(str))

# View the mapping
print(dict(zip(le.classes_, le.transform(le.classes_))))
```

Scikit-learn's LabelEncoder.fit_transform() performs two steps simultaneously: fit() learns the unique classes, and transform() converts them to integers. The mapping is deterministic (alphabetical order) and reproducible.

3.5 Train-Test Split

```
from sklearn.model_selection import train_test_split

# Prepare features and target for future ML modeling
X = df[["release_year", "type_encoded", "rating_encoded", "content_age"]]
y = df["year_added"]

# 80% training, 20% testing, random_state ensures reproducibility
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

print(f"Training set: {X_train.shape}")   # ~7045 rows
print(f"Test set:     {X_test.shape}")     # ~1762 rows
```

train_test_split() divides the dataset into training and testing subsets. The **test_size=0.2** parameter means 20% of data is held out for evaluation. **random_state=42** fixes the random seed so results are reproducible across runs.

4. Experiment 16 — Line, Bar, Histogram & Scatter Charts

Experiment 16 covers foundational chart types using both Matplotlib and Seaborn. Each chart type is demonstrated in both its basic and advanced form side-by-side. These are the building blocks of visual data analysis.

4.1 Line Charts — Netflix Titles Per Year

Line charts are ideal for showing trends over time. The basic line chart shows total titles released per year, while the advanced version breaks down by content type.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Set global style for all plots
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams.update({
    "figure.dpi": 120,
    "axes.titlesize": 14,
    "axes.labelsize": 12
})

# Prepare data: count titles per release year
year_df = df["release_year"].value_counts().sort_index().reset_index()
year_df.columns = ["Release_Year", "Count"]
year_df = year_df[year_df["Release_Year"].between(2012, 2021)]

# Basic line chart
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
ax1.plot(year_df["Release_Year"], year_df["Count"],
         marker="o", color="steelblue", linewidth=2, markersize=7)
ax1.set_title("Netflix Titles Added Per Year")
ax1.grid(True, linestyle="--", alpha=0.6)

# Advanced: Movies vs TV Shows per year
type_year = df.groupby(["release_year","type"]).size().unstack(fill_value=0)
ax2.plot(type_year.index, type_year["Movie"],
         marker="o", color="royalblue", label="Movies")
ax2.plot(type_year.index, type_year["TV Show"],
         marker="s", color="tomato", label="TV Shows")
ax2.legend()
plt.tight_layout()
plt.show()
```

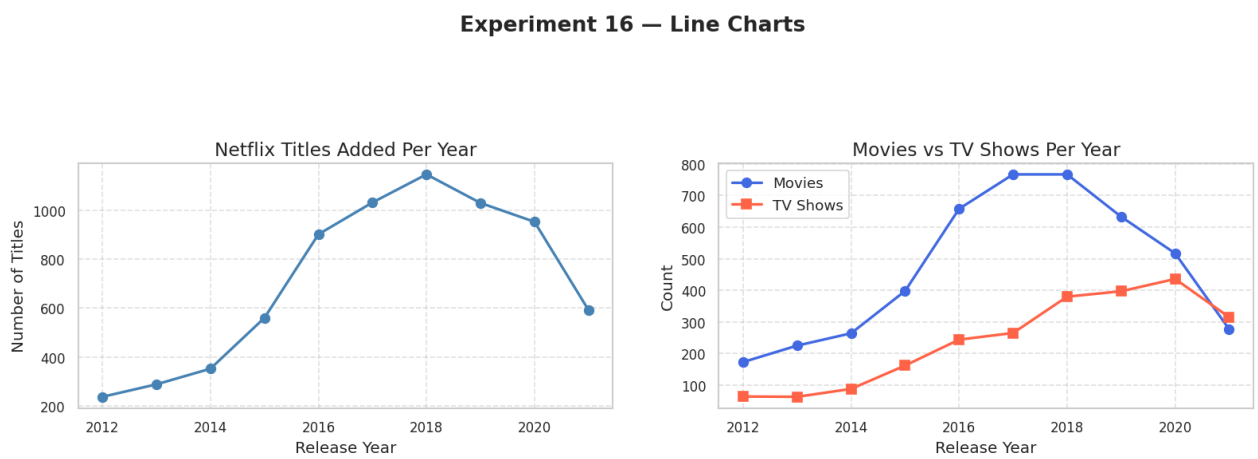


Figure 4.1 — Line Charts: Netflix Titles Per Year (basic) and Movies vs TV Shows Per Year (advanced)

Key insight: Netflix content additions peaked in **2018**, reflecting aggressive global expansion. After 2019, additions declined — possibly due to COVID-19 production slowdowns and strategic content curation.

4.2 Bar Charts — Rating Distribution

Bar charts compare quantities across discrete categories. The advanced version adds value labels on top of each bar for immediate readability.

```
# Prepare rating data
rating_df = df["rating"].value_counts().head(8).reset_index()
rating_df.columns = ["Rating", "Count"]

# Advanced bar chart with value labels
colors = plt.cm.Set2.colors  # Use a colormap for distinct bar colors
bars = ax2.bar(rating_df["Rating"], rating_df["Count"],
               color=colors[:len(rating_df)],
               edgecolor="white", linewidth=0.8)

# Add count labels above each bar
for bar in bars:
    y = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width()/2, y + 5,
             str(int(y)), ha="center", va="bottom",
             fontsize=9, fontweight="bold")
```

Experiment 16 — Bar Charts

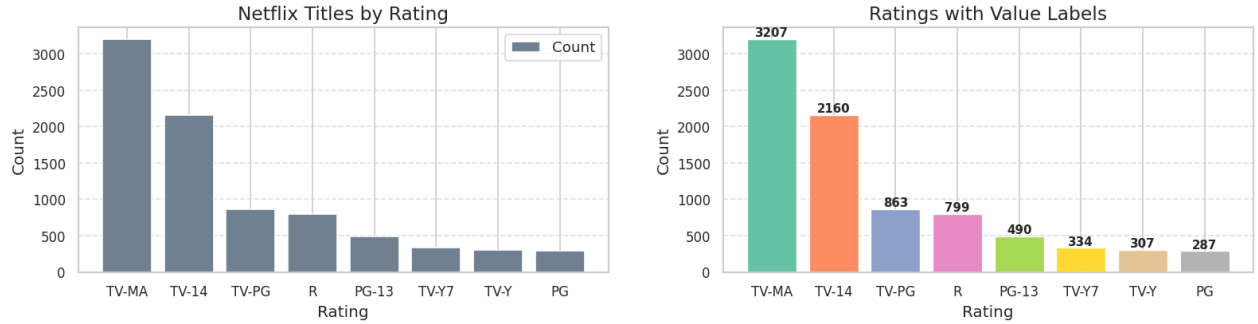


Figure 4.2 — Bar Charts: Netflix Titles by Rating (basic) and Ratings with Value Labels (advanced)

TV-MA (Mature Audience) dominates with 3,207 titles — over 40% of the dataset — confirming Netflix's primary focus on adult viewers. **TV-14** follows at 2,160 titles.

4.3 Histogram & Scatter Plots

Histograms visualize the frequency distribution of a continuous variable. Scatter plots show the relationship between two continuous variables.

```
# Histogram: Distribution of Release Years
ax1.hist(df["release_year"], bins=25,
         color="steelblue", edgecolor="white", linewidth=0.6)
ax1.set_title("Distribution of Release Years")

# Scatter: Release Year vs Movie Duration
movies["duration_min"] = movies["duration"].str.extract(r"(\d+)").astype(float)
ax2.scatter(movies["release_year"], movies["duration_min"],
            color="tomato", alpha=0.4, s=20, edgecolors="none")
ax2.set_title("Release Year vs Duration (Movies)")

# alpha=0.4: semi-transparent points to show density
# s=20: point size in pixels squared
```

Experiment 16 — Histogram & Scatter

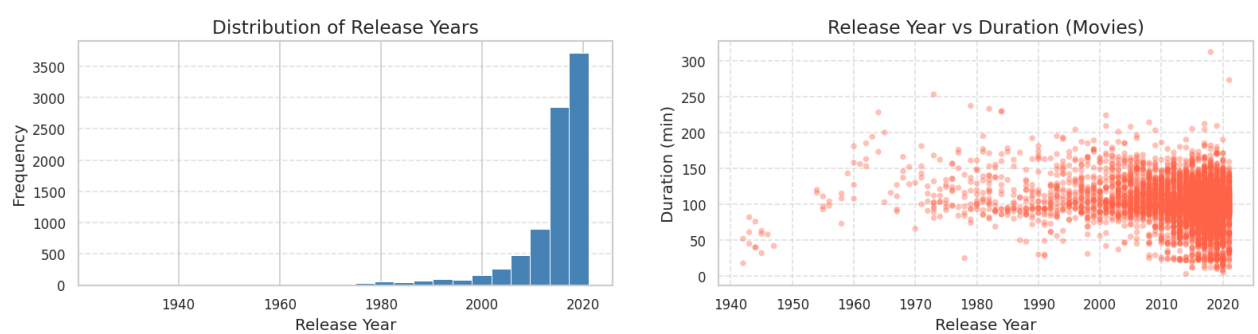


Figure 4.3 — Histogram (Distribution of Release Years) and Scatter Plot (Release Year vs Duration)

The histogram is heavily right-skewed — the vast majority of Netflix content was released after 2000, with a sharp spike after 2015. The scatter plot shows movie durations cluster tightly between 80–150 minutes regardless of era, confirming the industry standard feature-film format.

4.4 Conditional Scatter & Seaborn Line

```
# Conditional scatter: color by content type
colors_map = {"Movie": "royalblue", "TV Show": "tomato"}
for ctype, grp in sample.groupby("type"):
    ax1.scatter(grp["release_year"], grp["duration_min"],
                c=colors_map[ctype], label=ctype,
                alpha=0.5, s=25, edgecolors="none")
ax1.legend()

# Seaborn line plot (smoother API with confidence intervals)
sns.lineplot(x="Release_Year", y="Count", data=year_df,
             ax=ax2, color="steelblue",
             linewidth=2, marker="o")
```

Experiment 16 — Conditional Scatter & Seaborn Line

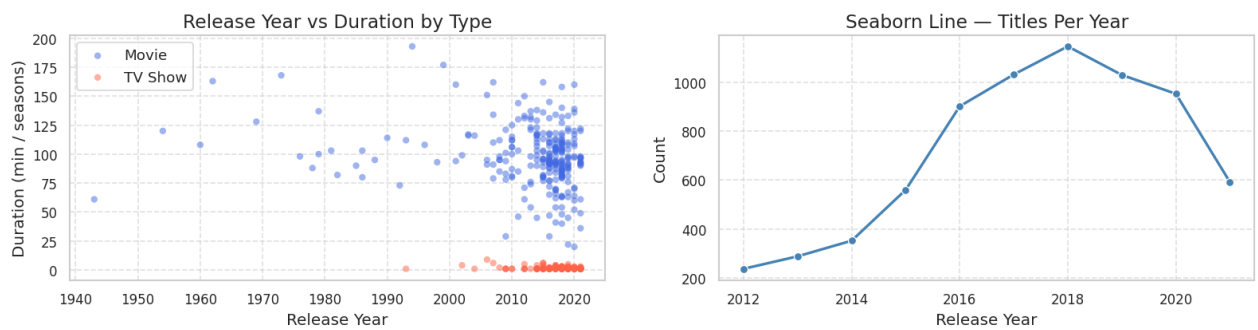


Figure 4.4 — Conditional Scatter (Release Year vs Duration by Content Type) and Seaborn Line Chart

4.5 Seaborn Histogram & Scatter

Seaborn provides a higher-level API over Matplotlib with built-in aesthetics and statistical features like kernel density estimation (KDE) overlay.

```
# Seaborn histogram with KDE
sns.histplot(df["release_year"], bins=20,
             color="slategray", ax=ax1, edgecolor="white")

# Seaborn scatter with automatic legend from "hue"
sns.scatterplot(x="release_year", y="duration_min",
                hue="type", data=sample, ax=ax2,
                alpha=0.5, s=30,
                palette={"Movie":"royalblue","TV Show":"tomato"})
```

Experiment 16 — Seaborn Plots

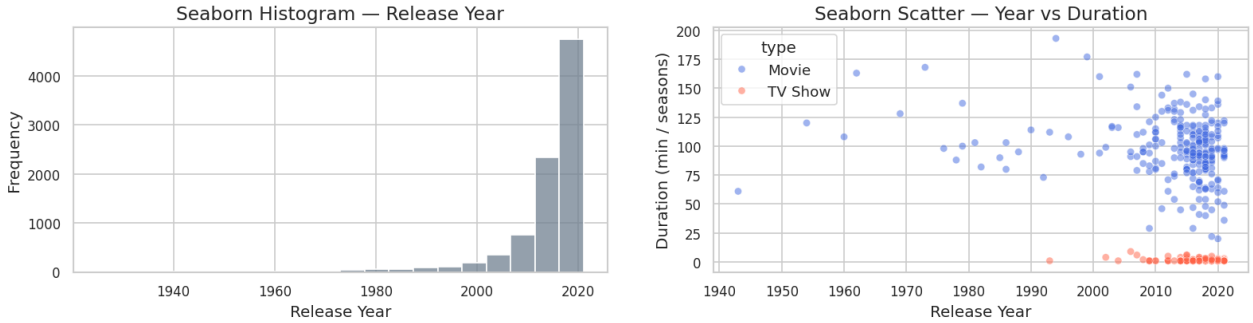


Figure 4.5 — Seaborn Histogram (Release Year) and Seaborn Scatter Plot (Year vs Duration by Type)

Seaborn's **hue** parameter automatically splits data by a categorical column and assigns colors, eliminating the need for manual groupby loops. TV Shows (red points) cluster near 0 on the y-axis because their "duration" is measured in seasons (1–10), while movies (blue) span 50–200+ minutes.

5. Experiment 17 — Area, Pie, Donut, Boxplot, Heatmap & Bubble

Experiment 17 introduces more sophisticated visualization types that go beyond simple comparisons to reveal proportions, distributions, correlations, and multi-dimensional relationships.

5.1 Area Plot & Pie Chart

```
# Area plot using fill_between
ax1.fill_between(year_df["Release_Year"], year_df["Count"],
                 color="royalblue", alpha=0.6)
ax1.plot(year_df["Release_Year"], year_df["Count"],
         color="royalblue", linewidth=2)

# Pie chart for content type proportions
type_df = df["type"].value_counts().reset_index()
ax2.pie(type_df["Count"],
        labels=type_df["Type"],
        autopct="%.1f%%",
        colors=["#4e79a7", "#f28e2b"],
        startangle=90,
        textprops={"fontsize": 12})
```

Experiment 17 — Area Plot & Pie Chart

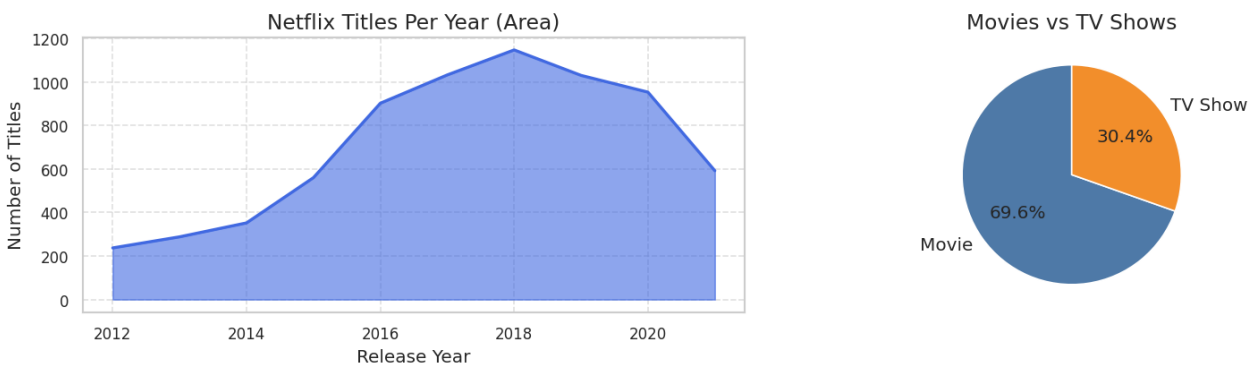


Figure 5.1 — Area Plot (Netflix Titles Per Year) and Pie Chart (Movies vs TV Shows)

The pie chart confirms that **69.6%** of Netflix content is Movies and **30.4%** is TV Shows. **autopct="%.1f%%"** formats the percentage labels automatically. The area plot's **fill_between()** creates a shaded region that emphasizes volume over time.

5.2 Donut Chart & Boxplot

```
# Donut chart: like a pie but with a hole in the center
ax1.pie(rating_df["Count"],
        labels=rating_df["Rating"],
        autopct="%.1f%%",
        colors=palette[:len(rating_df)],
```

```
        startangle=90,
        pctdistance=0.82,
        wedgeprops=dict(width=0.55, edgecolor="white", linewidth=2))

# width < 1.0 in wedgeprops creates the donut hole

# Seaborn boxplot: shows median, IQR, and outliers
sns.boxplot(x="type", y="release_year", data=df,
            palette={"Movie": "#4e79a7", "TV Show": "#f28e2b"},
            width=0.45, linewidth=1.5)
```

Experiment 17 — Donut Chart & Boxplot

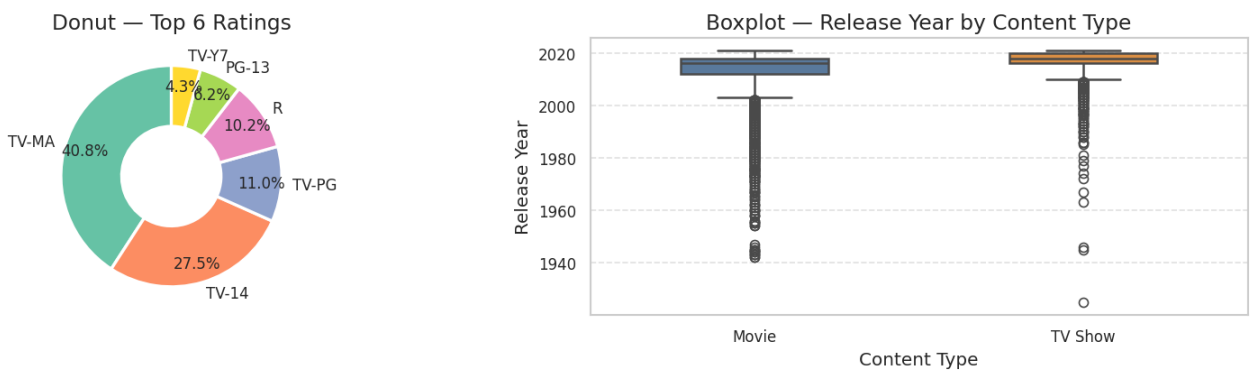


Figure 5.2 — Donut Chart (Top 6 Ratings) and Boxplot (Release Year by Content Type)

The boxplot reveals that both Movies and TV Shows have a median release year around 2018-2019, but Movies have a much wider spread of outliers going back to the 1940s, reflecting Netflix's inclusion of classic cinema. The interquartile range (IQR) — the box itself — represents the middle 50% of release years.

Key boxplot terminology: the center line is the median (50th percentile), the box spans Q1 to Q3 (25th to 75th percentile), whiskers extend to 1.5×IQR, and individual points beyond the whiskers are outliers.

5.3 Correlation Heatmap & Bubble Plot

```
# Compute correlation matrix for top 10 countries data
corr = top_countries[["Total", "Movies", "TV_Shows"]].corr()

# Heatmap with annotation
sns.heatmap(corr,
            annot=True,      # Show correlation values in cells
            cmap="coolwarm", # Red = positive, Blue = negative
            ax=ax1,
            linewidths=0.5,
            fmt=".2f",       # Format numbers to 2 decimal places
            annot_kws={"size": 11})

# Bubble plot: scatter where point SIZE encodes a third variable
ax2.scatter(top_countries["Movies"],
            top_countries["TV_Shows"],
```

```
            s=top_countries["Total"] * 0.6,  # Size = total titles
            c=range(len(top_countries)),      # Color = country rank
            cmap="tab10", alpha=0.75)
```

Experiment 17 — Heatmap & Bubble Plot

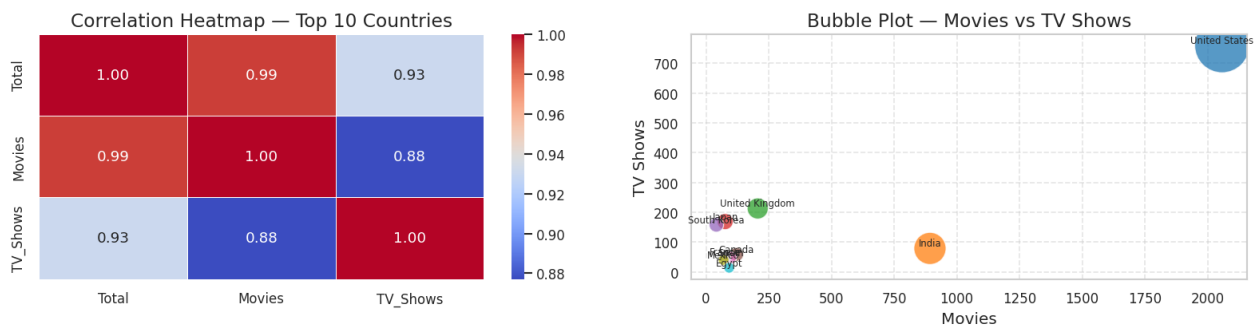


Figure 5.3 — Correlation Heatmap (Top 10 Countries) and Bubble Plot (Movies vs TV Shows per Country)

The heatmap shows near-perfect correlation (0.99) between Total and Movies across countries — countries with more content overall simply have more movies. The correlation with TV Shows is slightly lower (0.93) suggesting some countries specialize more in TV content.

5.4 Seaborn Bubble Plot

```
# Seaborn scatterplot with size as a third dimension
sns.scatterplot(
    x="Movies", y="TV_Shows",
    size="Total",           # Bubble size encodes total content
    hue="Country",          # Color encodes country
    data=top_countries,
    sizes=(80, 600),        # Min and max bubble pixel sizes
    palette="tab10",
    alpha=0.8,
    edgecolor="white")
ax.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
```

Experiment 17 — Seaborn Bubble Plot

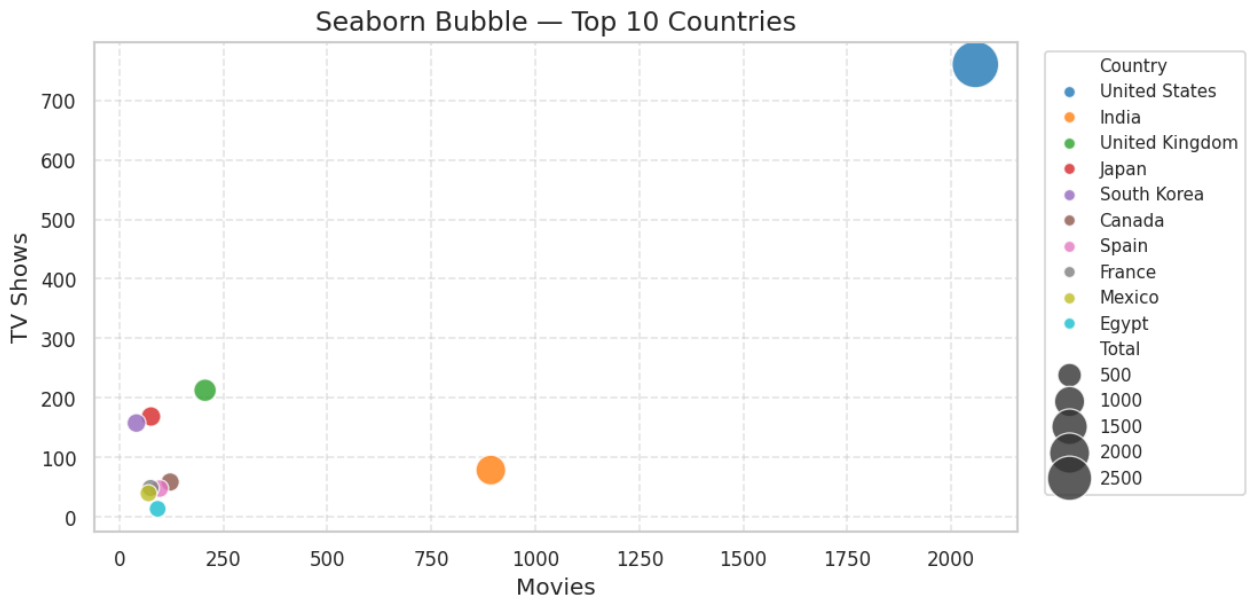


Figure 5.4 — Seaborn Bubble Plot: Top 10 Countries by Movies, TV Shows, and Total Content

The United States dominates with an enormous bubble in the top-right corner (2,000+ movies, 700+ TV shows). India has a large orange bubble representing its significant movie output (~900) but relatively few TV shows (~80). The sizes parameter in Seaborn's scatterplot maps numeric values to visual bubble sizes in the specified range.

6. Experiments 19 & 20 — Country Analysis & Temporal Trends

Experiments 19 and 20 perform deep-dive analysis on geographic distribution of Netflix content and temporal trends in content additions. These experiments use advanced data aggregation, multi-index groupby operations, and combined visualization techniques.

6.1 Country-Wise Data Aggregation

```
# Build country-type pivot table
country_type = (df.groupby("country")["type"]
                  .value_counts()
                  .unstack(fill_value=0)
                  .reset_index())

# Rename and add totals
country_type.columns.name = None
country_type = country_type.rename(columns={
    "Movie": "Movies", "TV Show": "TV_Shows"})
country_type["Total"] = country_type["Movies"] + country_type["TV_Shows"]

# Sort and get top 20
top = country_type.sort_values("Total", ascending=False).head(20)
```

.groupby().value_counts().unstack() is a powerful Pandas chain: groupby groups rows by country, value_counts() counts each content type, and unstack() pivots the inner index (type) into columns, creating a cross-tabulation (pivot table). **fill_value=0** replaces NaN with 0 for countries with no movies or TV shows.

6.2 Top 20 Countries Analysis

```
# Horizontal bar chart for top 20 countries
ax1.barh(top["country"][::-1], top["Total"][::-1],
         color="steelblue", edgecolor="white")
# [::-1] reverses order so highest appears at top

# Stacked bar chart: Movies vs TV Shows
x = range(len(top))
ax2.bar(x, top["Movies"], label="Movies",
        color="#4e79a7")
ax2.bar(x, top["TV_Shows"],
        bottom=top["Movies"],   # Stack on top of Movies
        label="TV Shows",
        color="#f28e2b")
ax2.set_xticks(list(x))
ax2.set_xticklabels(top["country"], rotation=45, ha="right")
```

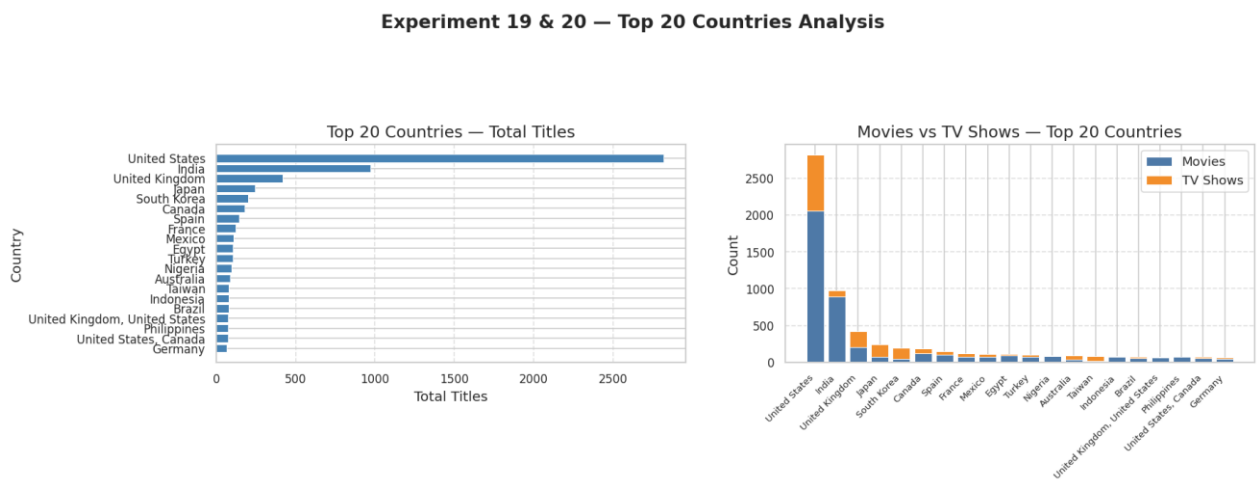


Figure 6.1 — Top 20 Countries: Total Titles (horizontal bar) and Movies vs TV Shows breakdown (stacked bar)

The horizontal bar chart uses **barh()** and the **[::-1]** slice reversal to display the USA at the top. The stacked bar chart's **bottom=** parameter stacks TV Shows on top of the Movie bars, making the total immediately visible while also showing the composition.

6.3 Content Age Distribution & Yearly Addition Trend

```
# Content age histogram
ax1.hist(df["content_age"].dropna(), bins=25,
         color="mediumseagreen", edgecolor="white")

# Yearly additions area chart
yearly = df.groupby(df["date_added"].dt.year).size().reset_index()
yearly.columns = ["Year", "Titles_Added"]

ax2.fill_between(yearly["Year"], yearly["Titles_Added"],
                 color="royalblue", alpha=0.5)
ax2.plot(yearly["Year"], yearly["Titles_Added"],
         color="royalblue", marker="o", linewidth=2)

# .dt.year extracts just the year component from a datetime column
```

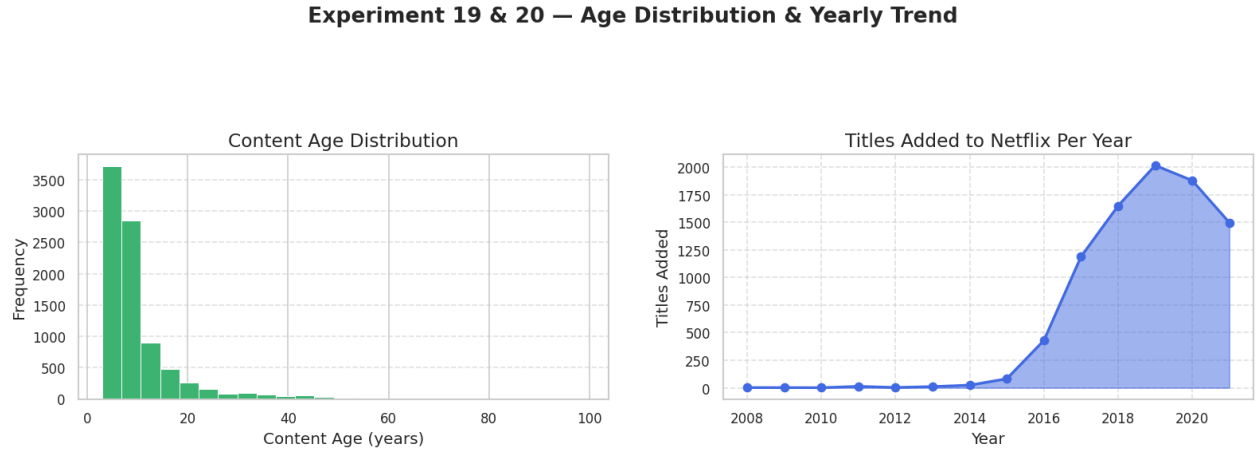


Figure 6.2 — Content Age Distribution (histogram) and Yearly Netflix Title Additions (area chart)

The content age histogram shows most Netflix content is 0–20 years old, meaning Netflix predominantly streams recent content. The long tail extending to 80+ years represents classic films from the 1940s-1950s. The yearly additions area chart reveals Netflix added over 2,000 titles in 2019 — its peak year — before tapering off.

7. Statistical & ML Concepts Applied

This project demonstrates a spectrum of statistical analysis methods, from basic summary statistics to advanced preprocessing pipelines. Each concept is explained below with its application to the Netflix dataset.

7.1 Univariate Analysis

Univariate analysis examines one variable at a time to understand its distribution, central tendency (mean, median, mode), spread (range, variance, standard deviation), and shape (skewness, kurtosis).

```
# Numeric univariate analysis
print(df["release_year"].describe())
# count, mean, std, min, 25%, 50%, 75%, max

# Categorical univariate analysis
print(df["type"].value_counts())
print(df["rating"].value_counts())

# Skewness of release_year
print(df["release_year"].skew())
# Positive value confirms right-skewed distribution
```

7.2 Bivariate Analysis

Bivariate analysis examines relationships between two variables — how they co-vary, correlate, or differ across groups.

```
# Numerical vs Numerical: correlation
print(df[["release_year", "content_age"]].corr())

# Categorical vs Numerical: group comparison
print(df.groupby("type")["release_year"].describe())

# Categorical vs Categorical: cross-tabulation
ct = pd.crosstab(df["type"], df["rating"])
print(ct.head())

# Visualize bivariate relationship
sns.boxplot(x="type", y="release_year", data=df)
```

7.3 Data Distribution Analysis

Understanding the distribution shape of data is critical before selecting statistical tests or ML algorithms. Key distribution metrics:

Metric	What It Measures	Netflix Application
Mean	Average value of the distribution	Average release_year ≈ 2014
Median	50th percentile — middle value	Median release_year ≈ 2018 (right-skewed)
Std Dev	Average spread around the mean	Large std in content_age (wide range)

Skewness	Asymmetry of distribution tail	release_year is right-skewed (newer content)
IQR	Q3 - Q1: middle 50% of data	Used in boxplots to identify outliers
Kurtosis	Sharpness of the distribution peak	High kurtosis = extreme outliers present

7.4 Categorical Encoding Techniques

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# Label Encoding (ordinal or binary categories)
le = LabelEncoder()
df["type_encoded"] = le.fit_transform(df["type"])
# Movie=0, TV Show=1

# One-Hot Encoding (for multi-class nominal categories)
# Creates separate binary column for each category
rating_dummies = pd.get_dummies(df["rating"], prefix="rating")
print(rating_dummies.head())
# Columns: rating_G, rating_PG, rating_R, rating_TV-14, ...

# When to use which:
# LabelEncoder: binary (2 values) or ordinal (natural order) data
# OneHotEncoder: nominal data with 3+ unordered categories
```

Label Encoding assigns integers 0, 1, 2, ... to categories. It is efficient but implies an ordinal relationship (e.g., 0 < 1 < 2). One-Hot Encoding creates a binary column for each category, avoiding false ordinal relationships, but increases dimensionality.

7.5 Visualization Library Comparison

Library	Level	Best For	Used In Project
Matplotlib	Low-level	Full control, custom layouts, subplots	Line, bar, hist, scatter, area, bubble
Seaborn	High-level	Statistical plots, themes, auto-coloring	Boxplot, heatmap, scatter, histogram
Pandas	Mid-level	Quick plots directly from DataFrames	df.plot(), df.value_counts().plot()
Plotly	Interactive	Web-based interactive charts and maps	Choropleth world map (Exp 19)

8. Key Findings & Insights

8.1 Content Type Distribution

- Movies dominate the Netflix library at ~69.6% vs. TV Shows at ~30.4%
- This ratio has remained roughly consistent year-over-year, though TV show proportion grew slightly after 2017
- Netflix invests in both formats but movies require less ongoing commitment than multi-season shows

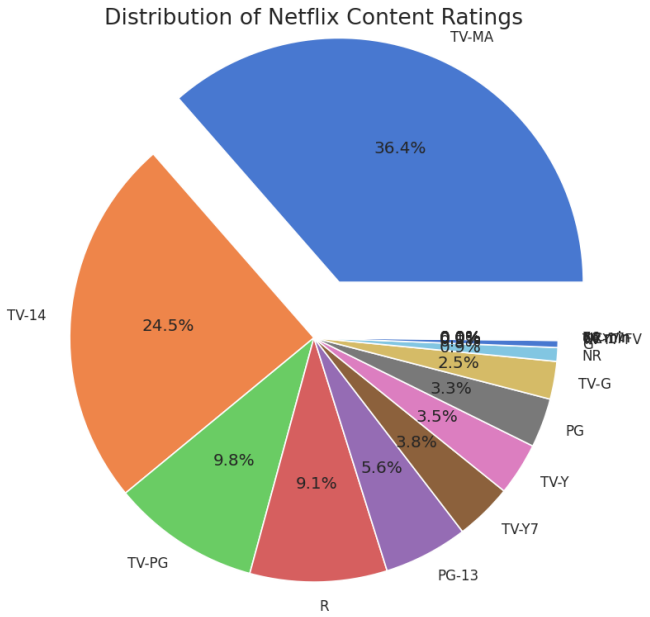
Using a Word Cloud To show most commonly occurring content genres

8.2 Geographic Content Production

- The United States produces the most content by a wide margin (2,800+ titles), followed by India (950+) and United Kingdom (400+)
- The US:India content ratio is approximately 3:1, reflecting Netflix's global expansion but also the growth of Bollywood-adjacent originals
- Emerging markets like Egypt, Nigeria, Turkey, and Mexico appear in the top 20, signaling Netflix's local content strategy

8.3 Content Ratings

- TV-MA (Mature Audience, 18+) is the single largest rating at 40.8% of all titles
- Combined adult ratings (TV-MA + TV-14 + R) account for ~79% of the library
- Netflix's content strategy clearly targets adult viewers despite having family-friendly offerings



Using Explode to highlight highest content rating type pie chart block

8.4 Temporal Trends

- Netflix's content additions grew exponentially from 2015–2019, peaking at ~2,000 titles added in 2019
- Post-2019 additions declined sharply — likely due to COVID-19 production halts and a strategic shift toward quality over quantity
- July and August are the peak months for adding new titles, suggesting strategic timing around summer viewership

8.5 Content Duration

- Most movies have a runtime between 90–120 minutes (the standard feature-length format)
- The majority of TV shows have only 1–2 seasons, indicating many are limited series
- Movie durations have remained stable across decades — no significant correlation with release year

8.6 Content Age

- Netflix primarily streams recent content: ~60% of titles are less than 10 years old
- The long tail of older titles (30–80+ years) represents classic cinema acquisitions
- The average content_age is approximately 8–10 years

9. Conclusion

This Exploratory Data Analysis of the Netflix dataset demonstrated the full EDA pipeline — from raw data loading and preprocessing through feature engineering, statistical analysis, and advanced visualization across four experiments.

The project covered 11 distinct chart types (line, bar, histogram, scatter, area, pie, donut, boxplot, heatmap, bubble, and stacked bar), each implemented in both basic and advanced forms using Matplotlib and Seaborn. Scikit-learn's LabelEncoder and train_test_split were applied to prepare the dataset for future machine learning tasks.

The analysis revealed that Netflix is primarily a platform for recent, adult-oriented content, with a heavy US and India production bias. The platform's content addition strategy peaked in 2018–2019 during its aggressive global expansion phase. These insights form a strong analytical foundation for building predictive models such as content recommendation engines or churn prediction systems.

9.1 Libraries & Tools Used

Library	Version	Purpose
pandas	2.x	Data loading, cleaning, groupby, pivot, datetime operations
numpy	1.x	Numerical arrays, mathematical operations, random sampling
matplotlib	3.x	Low-level plotting: figures, axes, subplots, all chart types
seaborn	0.13.x	High-level statistical plots with automatic aesthetics
scikit-learn	1.x	LabelEncoder, train_test_split, preprocessing pipelines
plotly	5.x	Interactive choropleth world map (Experiment 19)

End of Report — ENTC A2 | Netflix EDA Project